Cheat Sheet: MCP Server

Estimated time needed: 10 minutes

This module covered the basics of creating and configuring MCP Servers. Use this cheat sheet to quickly review and refresh the concepts you learned.

Transports

Overview

In MCP, the client and transport work together to carry messages between the AI application and the MCP server. The client acts as the translator and coordinator. It understands what the application needs, formats those requests correctly, and interprets the responses it receives. It also knows the MCP rules and ensures communication follows them.

The transport, on the other hand, is simply the path or communication channel that messages travel through, such as a WebSocket or a pipe. It does not understand the meaning of the messages. It simply delivers them back and forth reliably.

So, the client handles the logic and protocol, while the transport manages the low-level message delivery.

The Model Context Protocol (MCP) communicates using the JSON-RPC format. But how does this information travel between the client and the server? MCP supports two standardized transports: **STDIO** and **HTTP**.

STDIO

- **JSON-RPC messages** are exchanged over the system's **`stdin` (standard input)** and **`stdout` (standard output)** streams.
  - These streams are the data "pipes" of a program handled by the **operating system**.
- **STDIO** doesn't require a **network connection**, which makes it ideal for **local implementation and development**.
- Think of it as a direct "plug-in" connection between the **client and server** that uses **standard input and output streams** for communication.
  - The **client writes requests** to the server's `stdin`, and the **server sends responses** back through `stdout`.

HTTP

- **JSON-RPC messages** are exchanged using **HTTP** requests and responses — **HTTP** is the protocol used by **web browsers** to send and request information.
  - The **client sends** a JSON-RPC request in the body of an **HTTP POST** request, and the **server replies** with a **JSON-encoded response**.
- This method allows communication **over a network** (e.g., between machines, across the Internet), making it ideal for **remote MCP servers** with support for **authentication**, **encryption**, and **routing**.
- Think of it as sending and receiving structured "letters" over the Internet — each "letter" is an **HTTP request** containing a **JSON-RPC message** in its body.
  - The **client acts as the sender**, wrapping the message with **HTTP headers** (like the envelope's address and stamp).
  - The **server acts as the recipient**, reading the request, processing it, and sending back an **HTTP response** (another JSON "letter") with its own **headers and body**.
  - This **request–response cycle** follows **standard web rules**, allowing messages to travel **over the web**.

SSE (deprecated)

- **Server-Sent Events (SSE)** is a **deprecated transport** used for **streaming messages** from the server to the client over **HTTP**.
- It used a **unidirectional connection**, meaning the client could **receive continuous updates** but couldn't send data the same way.
- SSE relied on the **server keeping an open HTTP connection** to push updates.
- It has been **deprecated** in favor of **simpler and more reliable transports** (**STDIO** and **HTTP**), which support **full two-way communication**.
- Think of it as an old "radio broadcast" style — the **server could talk**, but the **client could only listen**.

Main components

MCP Servers provide three main components, also known as **primitives**, to clients: **tools, resources, and prompts.**

Tools

- Tools enable interactions with external systems; they are active functions that perform actions.
  - When something needs to do work or cause a change, use tools.
  - e.g., Query a database, call an API, perform computations
- Tools have unique name identifiers and metadata (description, parameters) that the client can read to understand their usage.
  - Tools have defined **input and output schemas** to ensure safe, consistent, and predictable behavior.
- Tools are designed to be **model-controlled**; this means that LLMs discover and invoke tools automatically based on their contextual understanding (metadata) and the user's prompting.

Resources

- Resources provide **passive** access to data; they expose structured information that the model can **read, reference, or observe**, but not directly modify.
  - Example: Reading from a document, fetching stored configuration data, or accessing project files.
  - Active data access is better handled by a **tool**.
- Each resource follows a **Uniform Resource Identifier (URI)** convention (e.g., `mcp://resource/secret`), allowing it to be uniquely identified and referenced by clients or tools.
- **Resource templates** define patterns for generating multiple related resources (e.g., `mcp://resource/{userId}/settings`).
- Resources are designed to be **application-driven**, meaning that host applications determine how to incorporate them based on its needs.
  - e.g., The application has set triggers to call a resource.

Prompts

- Prompts are **templates** that define reusable instructions, workflows, or conversation starters for the model.
  - Example: "Summarize this document," "Generate a report outline," or "Draft an email reply."
- Prompts contain structured input parameters that users or systems can fill in to guide the model's behavior.
  - Clients can discover available prompts, retrieve their contents, and fill them in with custom parameters.
- Prompts are designed to be **user-controlled**; they focus on guiding model reasoning rather than performing actions.

Context

MCP Context provides access to the active MCP session and its capabilities. This allows MCP primitives to interact with the server during execution. In **Module 2**, you encountered these three features of MCP Context:

- **Logging**: Record informational messages, warnings, or errors from within tools and resources for debugging or monitoring.
- **Progress reporting**: Communicate task progress or completion status back to the client in real time.
- **User elicitation**: Prompt the user for input or confirmation when additional information is needed during execution.

There are more capabilities of the MCP Context that you will explore in later modules.

FastMCP implementation

Instantiate your server object:

```
from fastmcp import FastMCP
mcp = FastMCP("MCP Server Name")
```

Define primitive MCP components (tools, resources, prompts):

```
@mcp.tool()
async def mcp_tool(input_parameter: str):
    # do something
    return "Return something"
@mcp.resource("uri://uniform/resource/identifier/{parameter}")
async def mcp_resource(parameter: str):
    # get resource
    return "The resource"
@mcp.prompt()
async def mcp_prompt(input_parameter: str):
    return f"Prompt template with {input_parameter}"
```

Use MCP Context:

```
from fastmcp import Context
from pydantic import BaseModel
class ElicitSchema(BaseModel):
    field: str
@mcp.tool()
async def context_tool(ctx: Context):
    await ctx.info("Logging message")
    await ctx.warning("Warning message")
    await ctx.error("Error message")
    await ctx.elicit(
        message="Please provide information",
        response_type=ElicitSchema
    )
    return "Tool return"
```

STDIO Manual Tool Calling:

```
from fastmcp.client.transport import StdioTransport
from fastmcp import Client
transport_stdio = StdioTransport(
    command="python",
    args=["path/to/server/script"]
)
stdio_transport_client = Client(transport_stdio)
async with client:
    tools = await client.list_tools()
    result = await client.call_tool("tool_name", {"tool_param": "param_value"})
```

OR using lower-level library `mcp`:

```
from mcp.client.stdio import stdio_client
from mcp import ClientSession, StdioServerParameters
server_params = StdioServerParameters(
    command="python",
    args=["path/to/server/script"]
)
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        tools = await session.list_tools()
        result = await session.call_tool(
            name="tool_name",
            arguments={"tool_param": "param_value"},
        )
```

HTTP Manual Tool Calling:

```
from fastmcp.client.transport import StreamableHttpTransport
from fastmcp import Client
transport_http = StreamableHttpTransport(
    url="url_of_http_mcp_server/mcp"
)
http_transport_client = Client(transport_http)
async with client:
    tools = await client.list_tools()
    result = await client.call_tool("tool_name", {"tool_param": "param_value"})
```

OR using lower-level library `mcp`:

```
from mcp.client.streamable_http import streamablehttp_client
from mcp import ClientSession
async with streamablehttp_client("url_of_http_mcp_server/mcp") as (read, write, _sid):
    async with ClientSession(read, write) as session:
        await session.initialize()
        tools = await session.list_tools()
        result = await session.call_tool(
            name="tool_name",
            arguments={"tool_param": "param_value"},
```

)

Author(s)

Joshua Zhou | Data Scientist Intern @ IBM
Joseph Santarcangelo | Data Scientist @ IBM